Two Axis Person Tracking Camera

Technical documentation and commissioning guide

Parker Wilson | Revision 1.0 | November 4, 2026

Project purpose

This project uses a USB camera and a pretrained YOLO person detector to steer a camera toward a person. Python measures the person's position in each image and sends pan and tilt angles to an Arduino Uno. Two position servos rotate the camera so the selected person approaches the image center.

Implementation status

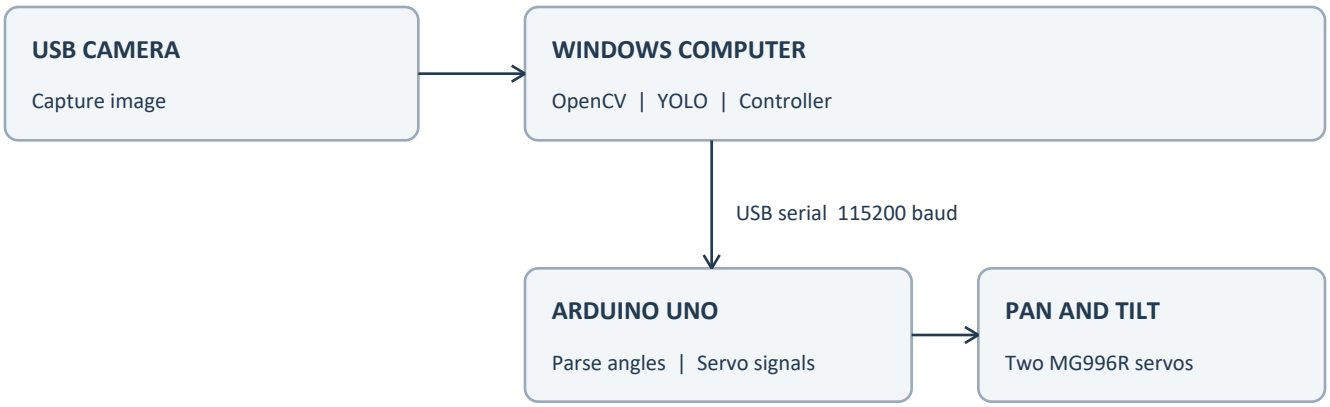
The project is an actively calibrated prototype. Parker reported that the standalone Arduino test moved both motors correctly and that pan responded during tracking. Supplied logs confirm person detections and changing serial angle commands, but also show a period at the 30 degree command limit. Consistent tracking on both axes after the latest changes has not yet been demonstrated in the available evidence.

Item	Documented baseline
Controller	Elegoo Uno R3 with direct servo outputs
Actuators	Two MG996R servos; owner confirmed 180 degree versions
Vision	Python, OpenCV, Ultralytics YOLO; model path yolo26n.pt
Home position	40,40 degrees, as most recently selected by the owner
Command limits	30 through 150 degrees on each axis
Interface	USB serial; ASCII pan,tilt followed by newline

How to use this guide

Wire and test the hardware first, upload the serial receiver firmware, then calibrate home position and direction signs. Use the validation checklist before treating the system as ready for demonstrations. The supplied firmware is a documentation reference, not a hardware-tested release.

System Architecture



Servo movement changes the next camera image, closing the visual feedback loop.

The computer performs image processing and control calculations. The Arduino generates servo signals independently of the frame-processing workload. No PCA9685 driver is required in the current design.

Component	Responsibility
Camera	Streams images from the moving camera assembly to the computer.
OpenCV	Captures frames, displays the annotated view, and handles keyboard events.
YOLO	Produces person bounding boxes for each frame using pretrained weights.
Python controller	Selects a box, smooths its center, computes pixel error, and updates commanded angles.
Arduino firmware	Reads newline-terminated angle pairs and writes accepted positions to pins 9 and 10.
External servo supply	Powers the servos; its ground is shared with the Arduino.

Project file roles

The submitted application imports calculate_direction from tracking.py. That helper supplies the direction labels shown on screen; the servo-motion calculation uses pixel errors directly. Keep main.py, tracking.py, and the configured model file accessible from the project environment.

The target is the first detected person box in each result. This is frame-by-frame detection without persistent person identity. With several people visible, the selected person can change between frames.

Hardware and Wiring

Disconnect power before changing wiring. The following connections describe direct Arduino control, which was the selected implementation after setting aside the servo driver board.

Connection	Destination
Pan signal, usually orange/yellow	Uno digital pin 9
Tilt signal, usually orange/yellow	Uno digital pin 10
Both servo positive wires, usually red	Regulated external servo supply positive
Both servo grounds, usually brown/black	External supply negative
Uno GND	External supply negative, shared with servo grounds
Uno USB	Computer USB port
Camera USB	Computer USB port

Power requirements

Use a regulated supply whose voltage is within the specifications of the actual servos and whose current rating supports both motors under load. A regulated 5V supply rated for 5 to 6A was recommended as a practical starting point for this two-servo assembly. Verify the actual servo and supply labels; marketplace MG996R variants can differ. [1, 2]

The photographed Elegoo breadboard module was used during development. Its documented maximum output is 700mA, so it should not be treated as a validated supply for two loaded MG996R servos. The owner later reported adequate power; no load-voltage measurement or replacement supply rating was provided. [3]

The module's 6.5 to 9V barrel-input rating describes its input, not a safe direct servo voltage. Never feed 9V directly to these servos. For the recommended arrangement, bypass that module and use suitable power wiring and connectors directly to the servo supply. Do not route the combined motor current through the Uno or small breadboard jumpers.

Keep the Uno powered by USB and connect the external supply negative to Uno GND. Do not connect the external positive to the Uno 5V pin in this arrangement. Secure the base, leave cable slack, and stop a test if the mechanism binds or a servo heats abnormally.

Software Setup

Use one Python environment for installation and execution. The examples below assume Windows PowerShell and a project folder containing main.py and tracking.py. Package versions were not captured as a reproducible lockfile during this session.

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install opencv-python ultralytics pyserial
```

Use a Python version supported by those packages. If package installation fails, follow the package's supported interpreter requirements rather than assuming compatibility with every Python release.

Confirm the camera and model

The provided script uses camera_index = 0, confidence_threshold = 0.5, image_size = 320, and classes=[0]. Confirm the camera index opens the mounted USB camera rather than a laptop camera. Confirm the loaded model maps class 0 to person. [4]

The configured weights filename is yolo26n.pt. Keep that exact file available or set model_path to the actual weights location. This documentation does not claim that a filename alone guarantees availability, provenance, or compatibility. The project runs inference with pretrained weights; no custom model training was shown.

Install and upload the firmware

In Arduino IDE, select Arduino Uno and the correct USB port. Install the official Servo library through Library Manager if needed. Open the included firmware_reference folder and upload firmware_reference.ino. This receiver is different from the standalone movement-test sketch: the test moves by itself; the receiver waits for serial commands. [1]

Run the application

```
.\.venv\Scripts\python.exe -m serial.tools.list_ports
.\.venv\Scripts\python.exe main.py
```

Set the Python serial port to the returned Arduino port; COM5 was the submitted setting. Both sides use 115200 baud. Close Serial Monitor before running Python, and stop Python before uploading firmware or opening Serial Monitor.

Detection and Control Algorithm

Detect and smooth the target

For each successful camera frame, YOLO detects people. The application selects person_boxes[0] and calculates its center from the left, top, right, and bottom bounding-box coordinates. Result box coordinates refer to the original image; use the captured frame dimensions to find the image center. [4]

```
detected_x = (left + right) / 2
detected_y = (top + bottom) / 2
smoothed_x += alpha * (detected_x - smoothed_x)
smoothed_y += alpha * (detected_y - smoothed_y)
```

The first detection initializes the smoothed coordinates directly. When no person is detected, the target and smoothing state are cleared. The submitted alpha is 0.3. Lower alpha reduces sudden changes but adds lag as the camera itself moves.

Convert image error into motion

```
error_x = target_x - frame_width / 2
error_y = target_y - frame_height / 2
speed = clamp(gain * error, -max_speed, max_speed)
angle = clamp(angle + direction_sign * speed * dt, 30, 150)
```

Positive X error means the target is right of center. Positive Y error means it is below center because image Y increases downward. The direction signs must match the installed mechanism. Correct motion makes the magnitude of the image error decrease; increasing error is evidence to recheck direction.

The controller requests an angle update when at least 1/30 second has elapsed. Actual update rate is limited by frame processing. Elapsed time is capped at 0.1 second to avoid a large angle step after a delay. Commands are rounded to integer degrees before transmission.

Example: with X error +200px, gain 0.15 and a speed limit of 45 degrees/s, requested speed is +30 degrees/s. At dt = 0.033s, an axis with sign +1 increases by approximately 1 degree. This is a calculation example, not a measured response.

Each axis holds its position while its absolute pixel error is within the dead zone. The original 20px threshold allows small offsets intentionally. The later suggested 8px threshold is a tuning candidate, not a verified calibration result.

Configuration Reference

Setting	Original or observed	Latest guidance
Home pan and tilt	Originally 90; later 150	40,40 selected by owner; match both programs
Angle limits	30 to 150 on both axes	Keep matched in Python and Arduino
pan_sign	-1 in submitted code and supplied log behavior	+1 suggested as a test; verify on hardware
tilt_sign	+1 in submitted code	Verify independently with vertical movement
gain	0.15	0.15 remains a starting point
max_speed	90 degrees/s	45 degrees/s suggested for tuning
dead_zone	20 pixels	8 pixels suggested for tighter centering
alpha	0.3	Keep initially; tune after signs are correct
Camera and weights	0 and yolo26n.pt	Confirm selected camera and weights
Serial link	COM5; 115200 baud	Use actual port; retain matching baud

Home position is a mechanical calibration

Home is the shaft command that makes the installed camera face forward. It is not the image center and it is not necessarily 90 degrees. Set pan_angle and tilt_angle to 40.0 in Python and initialize both Arduino outputs to 40 before attaching the servos. This reduces mismatched startup commands; servo behavior during power-up still depends on the hardware.

With limits 30 to 150 and home 40, there are 10 command degrees available below home and 110 above it. This is software command headroom, not a measured physical field of view. If symmetric motion is required, mechanically align the horns near the middle of a verified travel range.

Tune one variable at a time

First verify home, then axis mapping, then signs. After those are correct, adjust gain and maximum speed for response, and dead zone and smoothing for steadiness. Do not compensate for a wrong sign by increasing speed, and do not widen limits without checking the mechanism.

Calibration and Commissioning

Verify serial movement

Stop Python. Open Serial Monitor at 115200 baud and select Newline. Keep the mechanism clear. Send one command at a time and wait for the motion to finish.

Command	Expected purpose
40,40	Move to the owner-selected forward position
45,40	Increase pan by 5 command degrees
40,40	Restore home
40,45	Increase tilt by 5 command degrees
40,40	Restore home

Choose direction signs from the mechanism

Observed motion when angle increases	Python sign
Camera pans right	pan_sign = +1
Camera pans left	pan_sign = -1
Camera tilts down	tilt_sign = +1
Camera tilts up	tilt_sign = -1

These sign rules assume an upright, unmirrored camera image and positive error defined as target minus image center. If image flipping or camera orientation changes, recalibrate. Assess the actual view, not only the movement of the servo horn.

Verify visual feedback

Close Serial Monitor and restart Python. Move right of image center and confirm the camera turns toward the selected person and X error shrinks. Repeat to the left. Then move above and below the center far enough to exceed the dead zone and verify Y error shrinks. Test one axis at a time before combining movement.

When an angle remains at 30 or 150, the software is saturated at a limit. A remaining image error does not mean inference has stopped. Check sign, physical home alignment, and remaining travel before changing the controller.

Operation and Serial Protocol

Control or status	Behavior in the submitted application
Space	Toggles tracking while the OpenCV window has keyboard focus
Q or close window	Exits the processing loop
TRACKING	Tracking enabled and a selected person is available
PAUSED	No tracking corrections; detection still runs
NO PERSON	No selected detection; current commanded angles are retained
Program exit	Camera and serial port close; OpenCV windows are destroyed

The Arduino continues generating servo signals after Python exits. Holding position can continue to draw current; closing the Python port does not turn off servo power. No automatic return home, search scan, or command watchdog is implemented in the documented baseline.

Command format

```
pan,tilt\n

Example ASCII line: 40,45
Python: arduino.write(b"40,45\n")
```

The included receiver accepts exactly two decimal integers separated by a comma, optionally followed by carriage return, and terminated by newline. Both values must be 30 through 150. Invalid, oversized, or out-of-range lines are ignored as a whole. This stricter reference parser consolidates the proposed receiver; deployment has not been hardware-verified. [5]

The link is one-way in this reference. Python does not receive an acknowledgment or measured shaft position. Angles printed in the terminal are requested positions, not encoder measurements. The servos internally control their shafts, while Python closes the outer feedback loop using camera images.

Useful diagnostics

```
print(
    f"{status} | Error=({error_x}, {error_y}) | "
    f"Angles=({pan_angle:.1f}, {tilt_angle:.1f})"
)
```

For longer runs, throttle diagnostics or record timestamps to a file; printing every frame can affect throughput. Displayed FPS in the original script excludes the later display/event-wait work, so it is an estimate of processing throughput rather than a full end-to-end latency measurement.

Validation Evidence and Test Plan

What has been demonstrated

Parker confirmed the standalone Arduino movement test worked correctly. Supplied logs show detected targets near (524,259), image center inferred as (320,240), X error near +204, and requested angles at (30,30). The log later shows pan commands increasing when X error becomes negative, consistent with the original negative pan sign.

The full retained log also contains segments near zero image error with pan around 37.3 and tilt at 30. These observations show detection and command updates, but the log alone cannot distinguish camera correction from person movement. It does not establish a settling time, accuracy distribution, or successful vertical tracking.

Acceptance tests to complete

Test	Pass condition	Status
Manual pan and tilt	Each axis moves independently and holds	Owner reported pass
Home alignment	40,40 produces forward view without collision	Owner selected; recheck
Horizontal feedback	X error decreases from both sides	Pending final validation
Vertical feedback	Y error decreases from above and below	Pending
Pause and target loss	No new angle correction while paused or absent	Implemented; test pending
Command validation	Malformed and out-of-range lines cause no motion	Reference parser only
Travel limits	No commands outside agreed limits; no binding	Pending full sweep
Loaded endurance	No reset, voltage collapse, or abnormal heating	Not measured

Record a repeatable trial

Record interpreter/package versions, model identity, camera resolution, supply rating and measured loaded voltage, home/limits/signs, and lighting. Log timestamps, selected target center, error, requested angles, and status. Measure time to enter the dead zone and time spent there while a stationary person remains visible. Repeat on both axes and include failures.

No benchmark FPS, detection accuracy, or power margin is certified by this guide. The latest suggested configuration requires on-device validation.

Troubleshooting and Limitations

Symptom	Check next
Angles stay at 30 or 150	Limit saturation; direction sign; home alignment; remaining travel
Tilt stays still with Y error +19	Expected with original 20px dead zone; test a larger vertical offset
NO PERSON appears intermittently	Lighting, occlusion, person framing, model/class mapping and confidence threshold
Angles change but motors do not	Correct receiver sketch, port, baud, shared ground and newline framing
Camera moves away from target	Reverse only the affected sign after a small manual direction test
Startup points camera upward	Match Arduino and Python home settings; check horn alignment
Jitter or repeated restarts	Check power under load, wiring, binding; then reduce gain or increase dead zone
Serial port access denied	Stop the program currently holding the port; close Serial Monitor
No terminal messages during Arduino test	Read Serial Monitor at 115200; Arduino prints are not Python terminal output

Current limits of the design

The detector tracks the center of a person box, not a face or a named individual. Selecting the first box can switch people. Smoothed coordinates can lag when either the person or camera moves. Pixel-based gain changes behavior with capture resolution and camera field of view. Integer degree commands limit very fine positioning.

Keeping a person centered depends on visibility, detection reliability, control direction, mechanical travel, and available speed. The system cannot guarantee centering for every movement. When no person is visible, it holds the last command rather than searching.

Future improvements

After validating both axes, consider persistent target selection, separate gains per axis, dead-zone hysteresis, resolution-normalized errors, a visual travel-limit indicator, and timestamped measurements. A command acknowledgment and watchdog could improve diagnosis and define behavior after a connection loss. Treat these as future work, not implemented features.

References and Package Contents

Evidence basis

This guide is based on Parker's submitted Python script, Arduino sketches exchanged during setup, hardware photographs, stated 180 degree servo type and home position, and the supplied terminal log. The two retained text uploads contain the same log; they were not counted as independent trials. The latest on-disk application was not inspected, and proposed tuning changes are identified separately from verified results.

Technical references

[1] Arduino - Servo motor wiring and external power
https://docs.arduino.cc/learn/electronics/servo-motors/

[2] Tower Pro - MG996R manufacturer specifications
https://towerpro.com.tw/product/mg996R/

[3] Elegoo - Breadboard power module datasheet hosted by PGCC
https://pgccphy.net/1020/datasheets/ELEGOO%20Breadboard%20Power%20Supply%20Module.pdf

[4] Ultralytics - Prediction results and configuration
https://docs.ultralytics.com/modes/predict/

[5] pySerial - Serial API reference
https://pyserial.readthedocs.io/en/latest/pyserial_api.html

[6] Arduino - Resolve busy serial ports during upload
https://support.arduino.cc/hc/en-us/articles/4403365313810-If-your-sketch-doesn-t-upload

Included files

File	Purpose
Person_Tracking_Project_Guide.pdf	Formatted project documentation
PROJECT_DOCUMENTATION.md	Editable Markdown version of the guide
README.md	Package navigation and quick start
firmware_reference/firmware_reference.ino	Serial receiver with 40,40 home and 30 to 150 limits
CALIBRATION_RECORD.csv	Blank fields for recording actual hardware calibration

The package does not overwrite the existing project or include model weights. The reference firmware was reviewed for protocol consistency but was not compiled or uploaded to the hardware in this documentation task. Keep a copy of the working sketch before applying it.